

Exercise 1 : TL;DR & ELI

World War I (often called WWI or the Great War) was a global conflict that lasted from 1914 to 1918. It involved many of the world's great powers and reshaped borders, governments, and international relations.

How it started

The immediate trigger was the assassination of Archduke Franz Ferdinand on June 28, 1914. He was the heir to the Austro-Hungarian Empire.

But the deeper causes had been building for years:

- **Militarism** – European powers built large, competitive armies and navies
 - **Alliances** – Two major blocs formed:
 - Triple Entente (France, Britain, Russia)
 - Triple Alliance (Germany, Austria-Hungary, Italy—though Italy later switched sides)
 - **Imperialism** – competition for colonies and global influence
 - **Nationalism** – strong ethnic and national tensions, especially in the Balkans
-

Major sides

Allied Powers

Included: Britain, France, Russia, later the United States (1917), Italy, and others.

Central Powers

Included: Germany, Austria-Hungary, Ottoman Empire, Bulgaria.

What the war was like







6

World War I is often remembered for:

- **Trench warfare** on the Western Front (France/Belgium)
- Massive casualties due to machine guns, artillery, and poison gas
- New technology: tanks, aircraft, submarines
- Extremely slow movement of battle lines despite huge losses

The war was not just in Europe—it also spread to Africa, the Middle East, and parts of Asia and the Pacific.

Turning points

- **1917: United States enters the war** on the Allied side

- **Russian Revolution (1917)** leads Russia to exit the war
 - Germany's final offensives fail in 1918
-

End of the war

The war ended on **November 11, 1918** with an armistice (ceasefire).

Peace was formalized in the Treaty of Versailles (1919), which:

- Forced Germany to accept blame for the war ("war guilt clause")
 - Required reparations payments
 - Limited Germany's military
 - Redrew many European borders
-

Aftermath and importance

World War I had huge consequences:

- Collapse of empires: German, Austro-Hungarian, Ottoman, Russian
 - Creation of new countries in Europe and the Middle East
 - Formation of the League of Nations (early attempt at global peacekeeping)
 - Economic and political instability that helped lead to World War II
-

If you want, I can break it down further (like a simple timeline, key battles, or why trench warfare was so dominant).

TL;DR World War 1

World War I was a global war from 1914 to 1918 sparked by the assassination of Archduke Franz Ferdinand, but really caused by long-term tensions in Europe (alliances, militarism, nationalism, and imperial competition).

Two major sides fought:

- **Allied Powers** (Britain, France, Russia, later the U.S.)
- **Central Powers** (Germany, Austria-Hungary, Ottoman Empire)

It was known for brutal trench warfare, massive casualties, and new military technology like machine guns, tanks, and poison gas.

The war ended in 1918 with an armistice, and the Treaty of Versailles formally ended it in 1919, reshaping borders and setting the stage for future conflicts like World War II.

ELI-7

Okay, here's **World War I (ELI-7 version)**:

A long time ago, many countries in Europe were like big rival teams that didn't trust each other.

One day, a very important prince (Archduke Franz Ferdinand) was killed, and that set everything off like a huge chain reaction.

So the countries split into two big groups and started fighting:

- One side: Britain, France, Russia (later the U.S. joined)
- Other side: Germany, Austria-Hungary, and friends

They fought a massive war from **1914 to 1918**.

Instead of fast battles, most of the fighting was in muddy trenches, and many soldiers got hurt because the weapons were very powerful.

In the end:

- The first group won
- The war stopped in 1918
- Many countries changed or broke apart afterward

Basically:

A small event helped start a huge war between countries that already didn't get along.

Exercise 2: COT

Structured Reasoning to generate Test Strategy document for Accessibility Testing

Context

Act as a Experienced Mobile Banking Test Manager

Instruction

Design a Test strategy document for Accessibility testing on a Mobile Banking app

[MANDATORY] Ask me 5 valid multiple choice questions one by one

[STRICTLY] ONCE AFTER ANSWERING THE FIRST QUESTION Go with next question

[CRITICAL]Once after answering all questions generate Test strategy document in .docx format

It's a pleasure to step into the role of your Mobile Banking Test Manager. Ensuring a banking app is inclusive isn't just about compliance; it's about financial dignity for all users.

To tailor this strategy document to your specific environment and the nuances of the 2026 mobile landscape, I need to gauge your current project scope.

Let's start with the first question:

1. Which primary accessibility standard will serve as the foundation for our Mobile Banking App's compliance audit?

- A) WCAG 2.2 (Level AA)
- B) Section 508 (US Federal standard)
- C) EN 301 549 (European standard)
- D) ADA Title III guidelines only

A

Excellent choice. WCAG 2.2 Level AA is the gold standard for modern mobile applications, especially with the newer success criteria regarding touch targets and dragging movements.

2. Which platform-specific assistive technologies will be prioritized for our primary screen reader testing?

- A) Android: TalkBack / iOS: VoiceOver
- B) Android: Select to Speak / iOS: Spoken Content
- C) Third-party screen readers downloaded from the App Store
- D) Emulated screen readers via Chrome DevTools only

B

Interesting choice. While TalkBack and VoiceOver are the standard native screen readers, focusing on **Select to Speak** and **Spoken Content** suggests we are targeting users who may have learning disabilities or low vision rather than total blindness.

3. Regarding the banking app's security features, how should the strategy handle "Timed Sessions" (Auto Logout) to satisfy accessibility requirements?

A) Disable session timeouts entirely for users with accessibility settings enabled. B) Provide a warning and allow the user to extend the session at least 10 times. C) Strictly enforce the 2-minute logout without warning to maintain high-level security. D) Implement a "Keep me logged in" checkbox that bypasses all security timeouts.

C

That is a common point of tension between security and accessibility. While security is paramount in banking, **WCAG 2.2 Success Criterion 2.2.1** generally requires that users be allowed to turn off, adjust, or extend the time limit unless it's an "essential" exception. Implementing a strict logout without warning can create significant barriers for users who need more time to process information or navigate the UI.

4. How will the strategy address "Color Contrast" for the banking app's brand-specific UI components (like the 'Transfer Funds' button)?

A) Brand colors are exempt from contrast requirements to maintain marketing consistency. B) Text must have a contrast ratio of at least 4.5:1; icons and graphical components must be at least 3.0:1. C) Contrast is only tested for the login screen; all internal transaction screens are exempt. D) Use "High Contrast Mode" as the only way to meet the requirement, leaving the default theme as-is.

A

That is a bold approach. While brand identity is vital, WCAG 2.2 is quite strict: brand-specific colors are not exempt if they are used for functional text or essential UI components. We may need to propose a "brand-compliant" secondary palette to ensure we don't exclude users with low vision or color blindness.

Last question before I compile the full strategy:

5. For the mobile banking app's "Biometric Authentication" (FaceID/TouchID), what is the mandatory accessibility fallback?

A) A physical signature submitted via the mobile camera. B) At least one non-biometric alternative (e.g., PIN, Password, or Pattern) that is also accessible. C) A phone call to a bank representative to verify identity. D) There is no fallback required; biometrics are considered inherently accessible.

B

Perfect. Providing an accessible fallback for biometrics is non-negotiable for a truly inclusive banking experience.

As your Mobile Banking Test Manager, I have synthesized your requirements and quiz responses into a formal **Accessibility Test Strategy**. Since I cannot directly "attach" a .docx file, I have generated the complete content below in a structured format that you can copy and paste directly into Word. It will maintain its professional hierarchy and clarity.

Test Strategy: Mobile Banking Accessibility

Project: NextGen Mobile Banking Application

Version: 1.0

Standards: WCAG 2.2 Level AA

Status: Draft for Stakeholder Review

1. Executive Summary

This document outlines the strategic approach for validating the accessibility of the Mobile Banking Application. The goal is to ensure that all customers, including those with vision, hearing, motor, or cognitive impairments, can securely and independently manage their finances.

2. Scope of Testing

2.1 In-Scope

- **Core Banking Journeys:** Login, Account Summary, Internal/External Transfers, Bill Pay, and Mobile Deposit.
- **Assistive Technology:** Platform-native tools (iOS Spoken Content, Android Select to Speak).
- **Visual Design:** Color contrast, text scaling, and non-text elements.
- **Interaction:** Touch targets, gesture alternatives, and session management.

2.2 Out-of-Scope

- Physical ATM accessibility.
- Third-party marketing websites linked from the app (unless mandated by contract).

3. Compliance Standards & Guidelines

- **Primary Standard:** WCAG 2.2 Level AA.
- **Banking Security Policy:** Adherence to strict session timeouts (C-level risk acceptance) and biometric fallbacks (PIN/Password).
- **Brand Policy:** Use of primary brand colors for UI components (Note: Risk of non-compliance identified).

4. Test Methodology

A hybrid approach combining automated scanning and manual expert review will be utilized.

4.1 Manual Testing (The Core)

- **Screen Reader Navigation:** Validating logical focus order and meaningful announcements for balance updates.
- **Biometric Fallback:** Ensuring users who cannot use FaceID/TouchID can navigate to the PIN entry screen without friction.
- **Timed Sessions:** Testing the user's ability to complete a transfer within the strict security window.

4.2 Automated Testing

- Utilization of tools like **Axe-DevTools for Mobile** or **Accessibility Inspector (Xcode)** to catch baseline errors like missing labels or insufficient touch targets.

5. Technical Requirements & Environment

Platform	Assistive Technology	Devices
iOS	Spoken Content / VoiceOver	iPhone 14, 15, 16 (Latest iOS)
Android	Select to Speak / TalkBack	Samsung Galaxy S23, Google Pixel 8

6. Key Accessibility Scenarios

1. **The "Transfer" Test:** Can a user with "Select to Speak" enabled complete a high-stakes transfer without timing out?

- 2. **The "Contrast" Test:** Are "Transfer Funds" buttons legible under various lighting conditions despite brand color constraints?
- 3. **The "Error" Test:** If a transfer fails, is the error message announced immediately and clearly to the user?

7. Defect Management

- **Severity 1 (Critical):** Any "Blocker" where a user cannot complete a financial transaction (e.g., cannot click 'Submit').
- **Severity 2 (High):** Significant difficulty or lack of fallback for a major feature.
- **Severity 3 (Medium):** Minor WCAG non-conformance that does not stop the transaction.

8. Risks & Mitigations

Risk	Mitigation
Brand Colors vs. Contrast: Brand colors may fail 4.5:1 ratio.	Advocate for high-contrast "Toggle" or outline-based button styles.
Strict Timeouts: Users may fail to complete tasks.	Optimize UI for speed; ensure "PIN" fallback is extremely fast to access.

Structured Reasoning to generate Automation framework for Playwright

Context

Act as an Automation Architect to design the framework for Playwright Automation

Instructions

[MANDATORY] Ask me 5 multiple chice questions one by one

[MANDATORY] Once after answering the first question ,proceed with net

[MANDATORY] Generate Framework Structure First. after completing all the questions

[STRICT] Proceed Step by Step for framework implementation

[STRICT] Up on my approval proceed with the next step

As an Automation Architect, I will help you design a robust, scalable, and maintainable Playwright framework tailored to your specific needs. Before we dive into the code and folder structures, I need to define the architectural constraints.

Let's start with the first question:

1. Which Architectural Pattern will serve as the foundation for this framework?

A) **Classic Page Object Model (POM):** Strong separation of page locators/actions from test logic.

B) **App-Action Pattern:** Direct interaction with the application state (skipping some UI steps) for faster execution.

C) **Screenplay Pattern:** An actor-centered approach focused on tasks, goals, and interactions.

D) **Data-Driven Framework:** Heavily focused on externalizing all inputs (JSON/CSV) with generic test scripts.

Step 1: Project Initialization & Configuration Implementation

As the Automation Architect, I will now provide the foundational setup. This includes the project structure creation and the core configuration file that governs how your tests run across browsers and environments.

1.1 Directory Setup & Dependency Installation

First, we initialize the project and install the necessary libraries for Playwright and Allure.

Bash

```
# Initialize Node.js project
```

```
npm init -y
```

```
# Install Playwright and Allure reporter
```

```
npm i -D @playwright/test allure-playwright
```

```
# Install Playwright Browsers
```

```
npx playwright install
```

1.2 The Configuration Core (playwright.config.ts)

This file is the "brain" of your framework. It implements your chosen strategy: **Multi-browser execution, Global Setup for Authentication, and Allure Reporting.**

TypeScript

```
import { defineConfig, devices } from '@playwright/test';
```

```
export default defineConfig({
```

```
  testDir: './tests',
```

```
  fullyParallel: true,
```

```
  forbidOnly: !!process.env.CI,
```

```
  retries: process.env.CI ? 2 : 0,
```

```
  workers: process.env.CI ? 1 : undefined,
```

```
  // Reporting strategy: Allure + Built-in HTML
```

```
  reporter: [
```

```
    ['html'],
```

```
    ['allure-playwright', { outputFolder: 'allure-results' }]
```

```
  ],
```

```
  // Global Setup for Storage State (Authentication)
```

```
  globalSetup: require.resolve('./global-setup'),
```

```
  use: {
```

```
    baseURL: 'https://your-banking-app.com', // Replace with dynamic env URL later
```

```
    trace: 'on-first-retry',
```

```
    screenshot: 'only-on-failure',
```

```
    video: 'retain-on-failure',
```

```

// Reuse the authentication state created in global-setup
storageState: 'auth/user.json',
},

/* Configure projects for major browsers */
projects: [
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
  },
  {
    name: 'firefox',
    use: { ...devices['Desktop Firefox'] },
  },
  {
    name: 'webkit',
    use: { ...devices['Desktop Safari'] },
  },
],
});

```

1.3 Environment & Data Mapping (data/env.config.ts)

Based on your preference for **External Constants**, we create a mapping to switch URLs and credentials based on an environment variable.

TypeScript

```

export const ENV_CONFIG = {
  staging: {

```

```
    url: 'https://staging.banking-app.com',
    username: process.env.STAGING_USER,
    password: process.env.STAGING_PASSWORD
  },
  prod: {
    url: 'https://banking-app.com',
    username: process.env.PROD_USER,
    password: process.env.PROD_PASSWORD
  }
};

export const getEnv = () => {
  const env = process.env.TEST_ENV || 'staging';
  return ENV_CONFIG[env as keyof typeof ENV_CONFIG];
};
```

Step 1 is complete. We have the project backbone, the multi-browser engine configured, and the environment data mapping ready.

Step 2: Global Setup and Authentication State Management

In this step, we implement the "Login Once" strategy. We will create a `global-setup.ts` file that performs the login at the start of the entire test run, captures the browser cookies and local storage, and saves them to a JSON file.

This ensures that every test in your projects starts already logged in, drastically reducing execution time.

2.1 Creating the Global Setup (`global-setup.ts`)

This script runs once before any workers start. It uses the credentials we defined in our `env.config.ts`.

TypeScript

```
import { chromium, FullConfig } from '@playwright/test';
```

```
import { getEnv } from './data/env.config';
```

```
import fs from 'fs';
```

```
import path from 'path';
```

```
async function globalSetup(config: FullConfig) {
```

```
  const { url, username, password } = getEnv();
```

```
  const storagePath = 'auth/user.json';
```

```
  // Ensure the auth directory exists
```

```
  const dir = path.dirname(storagePath);
```

```
  if (!fs.existsSync(dir)) {
```

```
    fs.mkdirSync(dir, { recursive: true });
```

```
  }
```

```
  const browser = await chromium.launch();
```

```
  const page = await browser.newPage();
```

```
  try {
```

```
    console.log(`Logging into ${url} as ${username}...`);
```

```
    await page.goto(url);
```

```
    // Selectors should be updated based on your actual Banking App UI
```

```
    await page.fill('input[name="username"]', username!);
```



```
    await page.fill('input[name="password"]', password!);
    await page.click('button[type="submit"]');

    // Wait for a post-login element (e.g., Dashboard heading)
    // to confirm login was successful
    await page.waitForSelector('text=Account Summary');

    // Save storage state into a JSON file
    await page.context().storageState({ path: storagePath });
    console.log('Authentication state saved successfully.');
```

```
  } catch (error) {
    console.error('Global Setup Login Failed:', error);
    throw error;
  } finally {
    await browser.close();
  }
}
```

```
export default globalSetup;
```

2.2 Updating the Configuration

We previously pointed to this file in `playwright.config.ts`. Now, Playwright will automatically execute this logic and look for `auth/user.json` to inject into every browser context during the test run.

2.3 Verification of State

To verify this works, our future tests will skip the login page and navigate directly to secure pages:

TypeScript

// Example of how tests will look now:

```
test('Transfer money should be accessible', async ({ page }) => {  
  await page.goto('/transfer');  
  
  // No login steps required here!  
});
```

Step 2 is complete. Your framework now handles authentication globally and efficiently.

Next Step: Step 3 - Implementing the Base Page and Page Object Model (POM) Structure.

Step 3: Implementing the Base Page and Page Object Model (POM)

In this step, we establish the **Base Page**, which serves as the "parent" class for all other pages. This centralizes common actions (like clicking, waiting for elements, or taking screenshots) to ensure our code is DRY (Don't Repeat Yourself). We will then create a specific **Page Object** for the Banking Dashboard.

3.1 The Base Page (pages/base.page.ts)

This class wraps Playwright's native methods with custom logging or enhanced waiting logic.

TypeScript

```
import { Page, Locator } from '@playwright/test';
```

```
export class BasePage {
```

```
  readonly page: Page;
```

```
  constructor(page: Page) {
```

```
    this.page = page;
```

```
  }
```

```
  // Common navigation helper
```

```

async navigateTo(path: string) {
    await this.page.goto(path);
}

// Enhanced click with logging or auto-waiting
async clickElement(locator: Locator) {
    await locator.waitFor({ state: 'visible' });
    await locator.click();
}

// Reusable method for filling text
async typeText(locator: Locator, text: string) {
    await locator.waitFor({ state: 'visible' });
    await locator.fill(text);
}

// Helper to get text content for assertions
async getElementText(locator: Locator): Promise<string> {
    await locator.waitFor({ state: 'visible' });
    return (await locator.textContent()) || '';
}
}

```

3.2 The Dashboard Page (pages/dashboard.page.ts)

This extends the Base Page and contains locators and actions specific to the banking dashboard.

TypeScript

```
import { Locator, Page } from '@playwright/test';
```

```
import { BasePage } from './base.page';

export class DashboardPage extends BasePage {

  readonly accountBalance: Locator;

  readonly transferFundsButton: Locator;

  readonly recentTransactions: Locator;

  constructor(page: Page) {
    super(page); // Pass the page object to the BasePage

    this.accountBalance = page.locator('.balance-amount');

    this.transferFundsButton = page.locator('button:has-text("Transfer Funds")');

    this.recentTransactions = page.locator('.transaction-list-item');
  }

  async getBalance(): Promise<string> {
    return await this.getElementText(this.accountBalance);
  }

  async goToTransferFunds() {
    await this.clickElement(this.transferFundsButton);
  }
}
```

3.3 Why this matters?

By using this hierarchy, if the "Transfer Funds" button changes from a button to a link in the next release, you only update the locator in **one** place (the DashboardPage class) instead of updating hundreds of test files.

Step 3 is complete. We now have a scalable blueprint for adding any page in the application.

Next Step: Step 4 - Designing the Test Suite and Allure Reporting Integration. Shall we proceed to write our first E2E test using these Page Objects?

Step 5: CI/CD Integration and Framework Maintenance

As an Automation Architect, the final piece of the puzzle is ensuring these tests run automatically whenever code is pushed. We will use **GitHub Actions** for the pipeline and implement a **Maintenance Strategy** to keep the framework healthy as the banking app evolves.

5.1 GitHub Actions Workflow (.github/workflows/playwright.yml)

This configuration will trigger your tests on every "Push" or "Pull Request" to the main branch. It handles environment setup, authentication, and report publishing.

YAML

```
name: Playwright Tests
```

```
on:
```

```
  push:
```

```
    branches: [ main, master ]
```

```
  pull_request:
```

```
    branches: [ main, master ]
```

```
jobs:
```

```
  test:
```

```
    timeout-minutes: 60
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
```

```
      with:
```

```
        node-version: 20
```

- name: Install dependencies

run: npm ci

- name: Install Playwright Browsers

run: npx playwright install --with-deps

- name: Run Playwright tests

run: npx playwright test

env:

TEST_ENV: staging

STAGING_USER: \${{ secrets.STAGING_USER }}

STAGING_PASSWORD: \${{ secrets.STAGING_PASSWORD }}

- name: Upload Allure Results

uses: actions/upload-artifact@v4

if: always()

with:

name: allure-results

path: allure-results/

retention-days: 30

5.2 Framework Maintenance Strategy

Activity	Frequency	Description
Dependency Updates	Monthly	Run npm update to get the latest Playwright features and browser engines.
Locator Review	Per Sprint	Identify brittle CSS selectors and migrate to data-testid attributes where possible.

Activity	Frequency	Description
Flakiness Audit	Weekly	Review Allure reports for tests with high retry rates and optimize their "Wait" logic.
Auth State Refresh	As Needed	Update global-setup.ts if the login flow changes (e.g., new MFA requirements).

Export to Sheets

5.3 Summary of the Architecture

- **Pattern:** Page Object Model (POM) for clean abstraction.
- **Speed:** Storage State (Global Setup) to bypass repetitive logins.
- **Data:** Environment-driven configuration via env.config.ts.
- **Visibility:** Allure Reports for professional stakeholder communication.
- **Reliability:** Integrated into CI/CD for constant feedback.

Architecture Implementation Complete.

You now have a production-ready Playwright framework. You can start scaling by adding more classes to the /pages directory and corresponding specs to the /tests directory.

Would you like me to generate a **README.md** file for your project to help onboard other team members?